CE 879: Information Security Management and Engineering

Homework 3

Software Supply Chain Security Analysis Tool (`sca`)

Farhad Tavakolian — 404204805

July 11, 2026

Analyzed repository: `fastapi/fastapi`

1 Practical Execution

1.1 Repository selection

The tool was developed and evaluated against **FastAPI** (`https://github.com/fastapi/fastapi`), a high-profile Python web framework:

- dependency management files: `pyproject.toml`, `requirements*.txt`, and a `uv.lock` lockfile with the fully resolved dependency graph;
- clearly defined MIT license;
- large CI/CD surface: 23 GitHub Actions workflow files;
- 56 direct dependencies (runtime + optional + docs/test tooling), far more than the required 5;
- 7,473 commits of git history — a meaningful secret-scanning test.

Analyzed commit: `7cb06f360dd4` on `master` (run of 2026-07-11).

Note: the FastAPI repository itself contains no Dockerfile. The tool detects and reports this gracefully; Docker checks were validated against a deliberately vulnerable synthetic repository (§1.7).

The tool is **not** specific to this project: manifests and lockfiles are auto-discovered for 8 ecosystems (PyPI, npm, Maven, Gradle, Go, Cargo, Composer, RubyGems) and every stage degrades gracefully on missing data.

1.2 Component analysis (dependency extraction + SBOM)

Command: `python3 -m sca https://github.com/fastapi/fastapi`

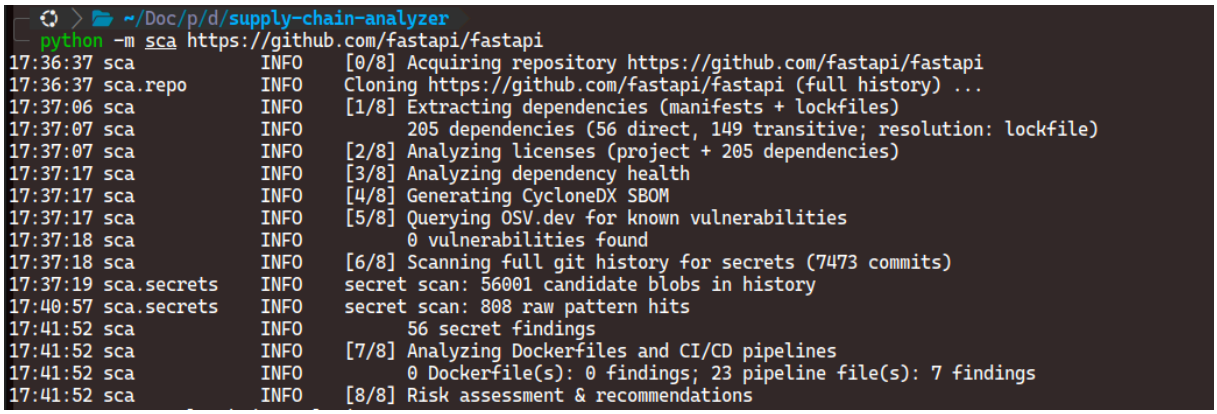


Figure 1: Console log of the eight-stage pipeline on `fastapi/fastapi` (run of 2026-07-11, 315 s total).

- **205 components** were identified: **56 direct**, parsed from `pyproject.toml` and `requirements*.txt` with scopes runtime, optional, dev and build, plus **149 transitive**, resolved exactly from `uv.lock` (`resolution: lockfile`).
- When a repository has no lockfile, the tool resolves the transitive closure online through the **deps.dev** resolved-dependency API (validated separately: 6 direct → 65 total components on a lockfile-less test repository), with a recursive PyPI `requires_dist` resolver as fallback. For Go projects the `// indirect` module graph recorded in `go.mod` is used directly (`resolution: go.mod`, validated on golang/go — §1.7).
- A **CycloneDX 1.5 SBOM** (`sbom.cyclonedx.json`) is generated with purl identifiers, versions, licenses, direct/transitive marking, and the dependency graph (root `dependsOn` = the 56 direct components).

1.3 Vulnerability analysis

Every resolved package version was checked against **OSV.dev** (batched queries, then per-advisory detail fetches). For FastAPI the result was **0 known vulnerabilities** — plausible for a flagship project whose lockfile is continuously updated by automation, and verified to be a true negative: the same scanner run against pinned old versions (`jinja2==3.1.2`, `express@4.17.1`) reports 17 vulnerabilities with CVE ids, CVSS v3.1 base scores (computed from the vector by the tool's own CVSS calculator), severity labels, descriptions and fixed versions — e.g. *CVE-2024-56201 (jinja2 sandbox breakout, HIGH, CVSS 8.8, fixed in 3.1.5)*. On the real-world golang/go run the scanner also surfaced *GO-2026-5932* (MEDIUM, `golang.org/x/crypto`: the unmaintained `openpgp` package).

1.4 Dependency health analysis

Documented metrics (registry metadata): **M1** days since latest release, **M2** releases in the last 365 days, **M3** total releases, **M4** deprecation/yanked flags, **M5** version lag of pinned dependencies. Classification thresholds: DEPRECATED (registry flag) / ABANDONED (>3 y) / STALE (18–36 mo) / AGING (6–18 mo, <2 releases/yr) / HEALTHY / UNKNOWN.

FastAPI result: **148 HEALTHY, 27 AGING, 19 STALE, 11 ABANDONED** — the abandoned set is mostly small, stable utilities of the docs toolchain (e.g. `jieba`, `mdx-include`, `webencodings`, `text-unidecode`), each reported with an actionable recommendation.

1.5 License analysis

- **Project license:** MIT (source: GitHub API; falls back to manifest declarations and LICENSE-file content heuristics offline).
- **Dependency licenses:** identified for 203/205 components from registry metadata (PEP 639 expressions → classifiers → free-text, npm license field, etc.), normalized toward SPDX.
- **Missing/unspecified:** 2 packages (`mypy-extensions`, `tiktoken`) — flagged for manual review.
- **Restrictive:** 4 weak-copyleft dependencies flagged (`cairosvg` LGPL-3.0-or-later; `certifi` and `pathspec` MPL-2.0; `tqdm` MPL-2.0 AND MIT) with compliance guidance.
- **Unusual/non-standard:** free-text or mixed declarations surfaced for review (`pygithub` free-text LGPL wording, `protobuf` "3-Clause BSD License", `regex` Apache-2.0 AND CNRI-Python).
- **Incompatibilities:** none. (Dual licenses such as `pytest-timeout`'s "DFSG approved OR MIT" are treated as a choice, so the permissive option applies. A GPL-only dependency inside this MIT project *would* be reported — validated on the synthetic repository.)

1.6 Repository security (secrets, CI/CD)

Secret detection scanned every unique blob across all **7,473 commits** of git history. 56 matches were found — **all classified as likely placeholders**: FastAPI's OAuth2/JWT tutorial ships an example `SECRET_KEY = "09d25e09..."` and sample JWT tokens replicated across `docs/*/docs/tutorial/security/oauth2-jwt.md` translations and `docs_src/security/tutorial00*.py`. Each finding still reports type, file:line, introducing commit, author/date, and ACTIVE/HISTORICAL status. True-positive behavior (a real GitHub PAT removed in a later commit → HISTORICAL; a live DB password → ACTIVE) was validated on the synthetic repository.

CI/CD analysis of the 23 workflows found FastAPI pins **every** action to a full commit SHA (best practice — zero pinning findings, a good true-negative test). Findings: **7 workflows triggered by** `pull_request_target` (labeling/triage automation; none checks out the PR head, so reported LOW with guidance).

1.7 Validation on other repositories

Because FastAPI is intentionally healthy, all detectors were additionally validated on a purpose-built vulnerable repository (old pinned dependencies, planted AWS/GitHub/DB-URL secrets — one later removed, a Dockerfile with `FROM python:latest` / no `USER` / secret in `ENV` / `curl | bash` / `EXPOSE 22`, and a workflow with `@main` action refs and `${{ github.event.pull_request.title }}` inside `run:`). The tool reported **CRITICAL (score 104.5)**: 17 vulnerabilities, 1 ACTIVE + 1 HISTORICAL secret, 7/7 Docker findings, 4/4 CI/CD findings, and the deprecated `left-pad` health flag.

A second real-world run against **golang/go** (66,853 commits, Go ecosystem, `resolution: go.mod`) exercised the non-Python path end-to-end: result **HIGH (score 35.5)**, driven by 4 likely-real secret matches — PEM private keys committed under `src/crypto/x509/` (in-tree test/platform fixtures, correctly ACTIVE/HISTORICAL-classified) and two historical token assignments in the long-deleted `misc/makerelease` tool — plus the OSV advisory GO-2026-5932 (§1.3) and 35 placeholder matches.

1.8 Risk assessment (FastAPI)

Overall: MEDIUM (score 18.5) — the full pipeline, including the full-history secret scan, completes in a few minutes (315 s on this run; Figure 1). The final console summary is shown in Figure 2.

Category	Contribution	Driver
vulnerabilities	0.0	none found
secrets	2.0 (capped)	56 placeholder tutorial values
cicd	4.0	7 × `pull_request_target` surface
docker	0.0	no Dockerfile
licenses	3.0	2 missing + 4 weak-copyleft deps
health	9.5	11 abandoned + stale transitive deps

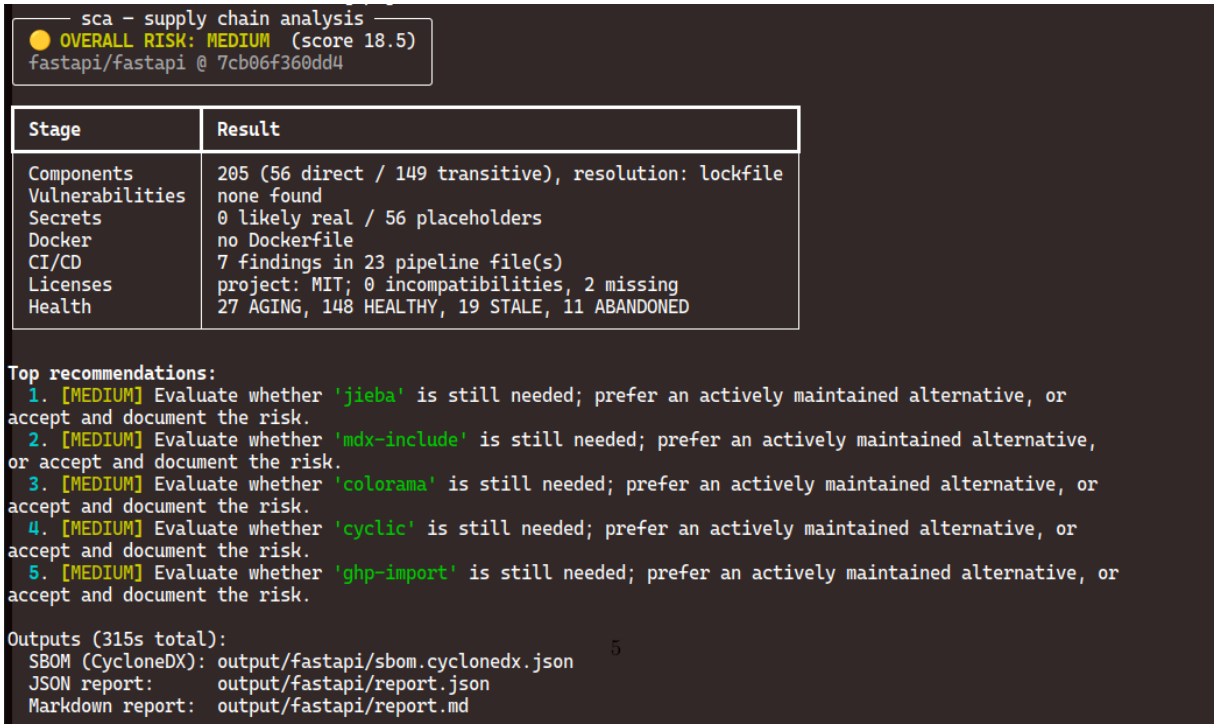


Figure 2: Final console summary for `fastapi/fastapi`: overall risk, per-stage results, and top recommendations.

Reasoning and one actionable recommendation per finding are generated automatically (see `output/fastapi/report.md`, sections 1 and 9).

2 Tools and Architecture

Language: Python 3.11+. **Libraries:** `requests` (HTTP + connection pooling/retries), `packaging` (PEP 440 requirement/version handling), `PyYAML` (workflows, pnpm/yarn-berry lockfiles), `rich` (console output, optional), stdlib `tomllib`/`xml.etree`/`configparser`/`subprocess`. Deliberately **no scanner frameworks** (no trivy/gitleaks/syft) — all analysis logic is implemented in the project.

2.1 External data sources and why

- **OSV.dev** — free, keyless, cross-ecosystem vulnerability database with exact version matching (`/v1/querybatch` + `/v1/vulns/{id}`); aggregates GHSA/PYSEC/CVE.
- **deps.dev** (Google Open Source Insights) — server-side dependency resolution used for the transitive closure when no lockfile exists; also metadata for Maven/Go.
- **PyPI / npm / crates.io / Packagist / RubyGems JSON APIs** — license metadata, release history (health metrics), latest versions.
- **GitHub REST API** — repository metadata + authoritative license id (anonymous; `GITHUB_TOKEN` optional).
- **OpenRouter API** (default model `anthropic/claude-sonnet-4.5`, overridable via `OPENROUTER_MODEL`; direct Anthropic API as fallback) — bonus executive summary only.

2.2 Pipeline

`sca/cli.py` orchestrates; each stage is isolated in its own module:

```
clone (full history) -> extract deps -> resolve versions/transitives
      |                                        |
      |                    licenses -- health +-> SBOM (CycloneDX 1.5)
      |                                        +-> OSV vulnerability scan
      +-> git-history secret scan (rev-list --objects -> cat-file --batch)
      +-> Dockerfile analysis
      +-> CI/CD workflow analysis
      +-------------> risk engine (scoring + reasoning + recommendations)
                          +-> report.json / report.md / console / LLM summary
```

Key design points: one shared HTTP session with retry/backoff and in-memory caching; `ThreadPoolExecutor` for registry fan-out; secret scanning streams blobs through a single `git cat-file -batch` process (no per-file exec); introducing commits located via `git log -find-object`; every parser is exception-isolated so malformed files become warnings, not crashes (a workflow with broken YAML, for example, still receives all text-based CI/CD checks).

2.3 Risk model

Severity weights: CRITICAL 10, HIGH 6, MEDIUM 3, LOW 1, INFO 0. Each category contributes a capped, severity-weighted sub-score:

Category	Scoring	Cap
Vulnerabilities	$\sum$ severity weights	40
Secrets	active real 12 · historical real 6 · placeholder 0.1 ($\leq$2 total)	40
CI/CD	$\sum$ severity weights	20
Docker	$\sum$ severity weights	15
Licenses	incompatibility 5 · strong copyleft 3 · weak copyleft 0.5 · missing 1	15
Health	deprecated 4 · not-on-registry 3 · abandoned 1.5 · yanked 2 · stale 0.25	15

Repeated findings of the same rule count 100% once, then 50% (N instances of one issue $\approx$ one process problem, not N independent risks). **Bands:** $<$10 LOW · $<$25 MEDIUM · $<$50 HIGH · $\geq$50 CRITICAL. **Floors:** an ACTIVE secret, a CRITICAL vulnerability, or a CRITICAL CI/CD finding each force at least HIGH.

3 Execution Guide

Prerequisites: Python ≥ 3.11, `git` in `PATH`, internet access (PyPI/npm/OSV/deps.dev/GitHub reachable).

```
unzip sca.zip && cd supply-chain-analyzer
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# run -- the only required argument is the GitHub URL
python3 -m sca https://github.com/OWNER/REPO
```

Outputs land in `output/<repo>/`: `sbom.cyclonedx.json`, `report.json`, `report.md` (plus a colored console summary).

Options: `-o DIR` output directory · `--skip-secrets` / `--skip-health` skip slow stages · `--local-path DIR` analyze an existing clone · `--keep-clone` · `-v` verbose · `--llm-summary` bonus executive report (requires `OPENROUTER_API_KEY`; or `ANTHROPIC_API_KEY` + `pip install anthropic` as fallback).

Bonus (§5.4 of the assignment): `--llm-summary` sends the condensed structured findings to an LLM through OpenRouter (default `anthropic/claude-sonnet-4.5`, any model selectable via `OPENROUTER_MODEL`) and writes `executive_summary.md` with a security posture summary, top risks, and a prioritized action plan.